

# Key-Value Store v0

## Versione studenti

Architetture dei Sistemi Distribuiti

Lezione 09

Alla fine di questa lezione dovrete saper distinguere chiaramente:

- interfaccia del servizio;
- contratto promesso al client;
- implementazione concreta del nodo;
- limiti che emergono appena il sistema cresce.

Punto chiave: la stessa API GET/SET/DELETE puo' avere semantiche molto diverse.

# Perche' partire da un KV store

Un key-value store sembra semplice, ma mette subito in evidenza problemi reali:

- modello dati minimale;
- operazioni intuitive;
- protocollo facile da testare;
- passaggio naturale da locale a distribuito;
- ottimo terreno per discutere correttezza e trade-off.

Non stiamo costruendo un database completo. Stiamo costruendo un caso di studio.

# Interfaccia vs implementazione

## Interfaccia

Che cosa puo' chiedere il client al servizio.

## Contratto

Quali garanzie riceve il client quando il servizio risponde.

## Implementazione

Come il server realizza quel contratto.

Errore classico: confondere la struttura dati locale con il servizio esposto in rete.

## Domanda guida

Che cosa significa davvero che una SET  $k \rightarrow v$  è andata a buon fine?

Nella versione di oggi significa solo:

- il processo server ha aggiornato il proprio stato in RAM;
- il server ha inviato OK sulla connessione;
- nessuna persistenza e nessuna replica sono implicite.

Questa domanda guiderà tutte le prossime evoluzioni.

- trasporto TCP;
- protocollo testuale UTF-8;
- una richiesta per riga;
- una risposta per riga.

```
PING
```

```
OK PONG
```

```
SET corso asd
```

```
OK
```

```
GET corso
```

```
OK asd
```

Il primo contratto del servizio include:

- PING: verifica di raggiungibilit ;
- SET key value: scrittura;
- GET key: lettura;
- DELETE key: rimozione;
- EXISTS key: presenza della chiave;
- KEYS: elenco delle chiavi;
- QUIT: chiusura della sessione.

Alcune decisioni sono piccole solo in apparenza:

- chiave assente: NOT\_FOUND oppure ERR?
- valore vuoto: va distinto da chiave assente?
- DELETE su chiave mancante: errore o no-op?
- KEYS: primitiva essenziale o comando diagnostico?
- ack di SET: successo locale o commit robusto?

Ogni scelta qui anticipa problemi che nel distribuito diventano costosi.



## Casi limite da discutere

```
GET corso  
NOT_FOUND
```

```
SET corso  
ERR usage: SET <key> <value>
```

```
SET corso ""  
OK
```

```
GET corso  
OK ""
```

Domanda: il protocollo distingue davvero tutti i casi che ci interessano?

# Che cosa promette davvero il v0

## Garanzie minime:

- richieste della stessa connessione processate in ordine;
- mappa chiave-valore coerente nel singolo processo;
- GET dopo SET sullo stesso nodo vede il valore scritto.

## Non garanzie:

- persistenza su disco;
- tolleranza ai guasti;
- replica;
- scalabilità;
- consistenza distribuita.

Lavoro richiesto:

- ① definire il contratto del protocollo;
- ② implementare il server TCP single-node;
- ③ gestire errori e casi limite;
- ④ testare manualmente il comportamento osservabile;
- ⑤ giustificare almeno una scelta semantica.

Durante il laboratorio chiedetevi sempre:

- che cosa promette davvero SET;
- come distinguere chiave assente e valore vuoto;
- quali errori sono di protocollo e quali applicativi;
- quali scelte locali oggi diventano problemi domani.

# Perche' KEYS e' didatticamente interessante

Oggi KEYS sembra innocuo:

- stato locale;
- costo basso;
- semantica ovvia.

Domani diventa ambiguo:

- su quale nodo?
- su quali repliche?
- su quali shard?
- con quale consistenza?

# Come evolve il percorso

Dopo il v0, la stessa interfaccia verra' messa sotto pressione:

- concorrenza e race condition;
- persistenza e crash recovery;
- replica primaria-secondaria;
- failover e leader election;
- quorum e conflitti;
- sharding e riequilibrio.

Ogni evoluzione dovra' aggiornare il contratto, non solo il codice.

## Idea centrale

Nei sistemi distribuiti l'interfaccia non basta. Conta il significato osservabile delle risposte.

Se non sapete spiegare cosa promette oggi OK, domani non potrete spiegare:

- cosa e' consistente;
- cosa sopravvive a un crash;
- cosa succede durante un failover;
- quali trade-off state accettando.